Integrating Temporal Memory into GraphCast: Architecture Overview and Alternative Approaches

Lianghong Mo

April 2026

Abstract

GraphCast is a state-of-the-art graph neural network for medium-range weather forecasting. It operates autoregressively but lacks explicit temporal memory across prediction steps. This document provides a self-contained overview of the GraphCast architecture, explains how we integrate a Mamba selective state space model to provide cross-rollout memory, presents our experimental findings (the approach does not yet outperform the baseline), and discusses alternative temporal memory mechanisms that may be more effective.

Contents

1 GraphCast: Architecture

1.1 The Weather Prediction Problem

Numerical weather prediction (NWP) aims to forecast the future atmospheric state given the current and recent observations. The atmosphere is described by a set of physical variables—temperature, wind components, humidity, pressure, geopotential—at discrete grid points across the globe and at multiple pressure levels (altitudes).

Traditional NWP solves partial differential equations (Navier-Stokes, thermodynamics) on a computational grid. GraphCast [1] replaces this with a learned mapping: given the atmospheric state at two consecutive time steps ($t-\Delta t$ and t, where $\Delta t = 6\text{h}$), predict the state at $t+\Delta t$.

1.2 The Encode-Process-Decode Framework

GraphCast follows an encode-process-decode structure, using an icosahedral mesh as a latent spatial representation:

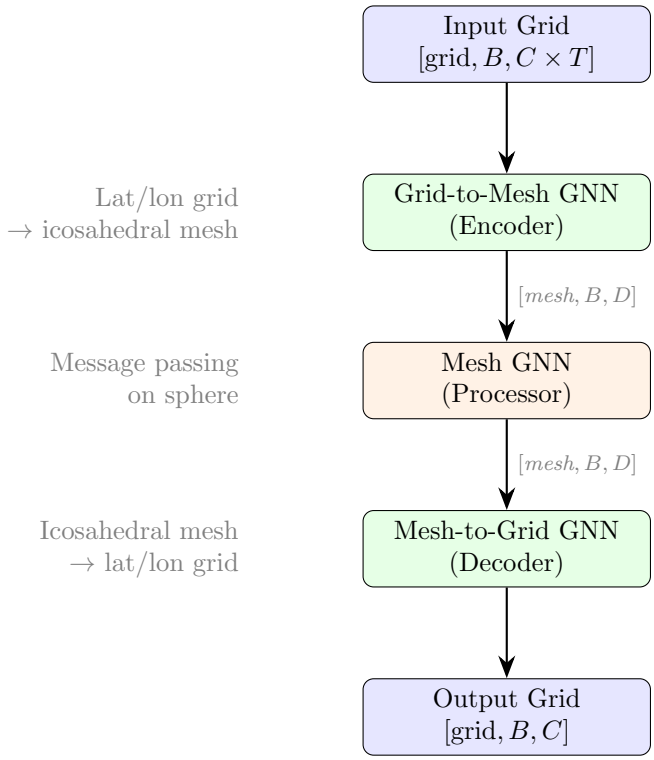

Figure 1: GraphCast's encode-process-decode pipeline for a single prediction step.

1.2.1 Step 1: Input Feature Construction

The input consists of T consecutive atmospheric snapshots (typically T=2). All time steps are **concatenated along the channel dimension**:

$$\mathbf{f}_i = [\mathbf{x}_{t-\Delta t}^{(i)} \; ; \; \mathbf{x}_t^{(i)}] \in \mathbb{R}^{2C}$$

where i indexes the N_{grid} latitude-longitude grid nodes. This simple concatenation allows a downstream MLP to learn temporal features such as:

- Velocity: $\mathbf{x}_t - \mathbf{x}_{t-\Delta t}$
- Acceleration (with T=4): $(\mathbf{x}_t - \mathbf{x}_{t-\Delta t}) - (\mathbf{x}_{t-\Delta t} - \mathbf{x}_{t-2\Delta t})$
- Any nonlinear combination of the concatenated frames

1.2.2 Step 2: Grid-to-Mesh Encoder

A graph neural network maps features from the regular lat/lon grid to an **icosahedral mesh**—a hierarchical triangulation of the sphere. The mesh provides:

- **Uniform spatial sampling**: Unlike lat/lon grids (which cluster at the poles), icosahedral meshes have approximately uniform node spacing.
- **Efficient global communication**: At coarse mesh levels, adjacent nodes span large geographic distances, enabling information to propagate globally in few message-passing steps.

The encoder performs one message-passing step on a bipartite graph connecting grid nodes to their nearest mesh nodes:

$$\mathbf{m}_j = \text{GNN}_{\text{encode}}\left(\{\mathbf{f}_i\}_{i \in \mathcal{N}(j)}\right) \in \mathbb{R}^D$$

where j indexes mesh nodes and $\mathcal{N}(j)$ are grid nodes within a query radius.

1.2.3 Step 3: Mesh Processor

The processor performs K rounds of message passing on the mesh graph (typically K=4 for the small model, K=16 for the full model). Each round updates mesh node features based on their mesh neighbors:

$$\mathbf{m}_j^{(k+1)} = \mathbf{m}_j^{(k)} + \text{MLP}\left(\mathbf{m}_j^{(k)}, \sum_{j' \in \mathcal{N}_{\text{mesh}}(j)} \text{MLP}([\mathbf{m}_j^{(k)}; \mathbf{m}_{j'}^{(k)}; \mathbf{e}_{jj'}])\right)$$

This enables global spatial information flow: with K=4 steps on a mesh with ∼2,500 nodes, information can travel from any point on the globe to any other.

1.2.4 Step 4: Mesh-to-Grid Decoder

A reverse GNN maps updated mesh features back to the lat/lon grid. The output is a residual prediction:

$$\hat{\mathbf{x}}_{t+\Delta t} = \mathbf{x}_t + \text{GNN}_{\text{decode}}(\mathbf{m}^{(K)})$$

1.3 Autoregressive Rollout

For multi-step forecasting, the prediction $\hat{\mathbf{x}}_{t+\Delta t}$ is fed back as input:

$$(\mathbf{x}_{t-\Delta t}, \mathbf{x}_t) \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+\Delta t} \xrightarrow{\text{feed back}} (\mathbf{x}_t, \hat{\mathbf{x}}_{t+\Delta t}) \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+2\Delta t} \to \cdots$$

Critical limitation: Each step sees only the two most recent frames. There is no mechanism to retain information from earlier steps. The model at step k has no knowledge of the original initial conditions or the intermediate trajectory—it sees only $(\hat{\mathbf{x}}_{t+(k-1)\Delta t}, \hat{\mathbf{x}}_{t+k\Delta t})$.

2 Mamba: Selective State Space Model

2.1 State Space Models

State space models (SSMs) are a class of sequence models derived from continuous-time linear systems:

$$\dot{h}(t) = Ah(t) + Bx(t) \tag{1}$$

$$y(t) = Ch(t) + Dx(t) \tag{2}$$

where $h(t) \in \mathbb{R}^H$ is the hidden state, $x(t)$ is the input, and $y(t)$ is the output. After discretization with step size Δ:

$$h_t = \bar{A}\, h_{t-1} + \bar{B}\, x_t, \quad \bar{A} = \exp(A \cdot \Delta) \tag{3}$$

$$y_t = C\, h_t + D\, x_t \tag{4}$$

The key property is that h_t is a **compressed summary of the entire input history** $x_1, x_2, \ldots, x_t$. Unlike Transformers, which require $O(T^2)$ attention over the full sequence, SSMs process sequences in $O(T)$ time with constant memory per step.

2.2 Mamba's Selective Mechanism

Standard SSMs use fixed A, B, C matrices. Mamba [2] makes B and Δ **input-dependent**, allowing the model to selectively remember or forget information:

$$\Delta_t = \text{softplus}(\text{Linear}(x_t)) \quad \text{(data-dependent step size)} \tag{5}$$

$$\bar{A}_t = \exp(A \cdot \Delta_t) \quad \text{(data-dependent decay)} \tag{6}$$

$$h_t = \bar{A}_t \cdot h_{t-1} + (1 - \bar{A}_t) \cdot x_t \tag{7}$$

When Δ_t is large, $\bar{A}_t \approx 0$ and the state resets to the current input ("remember this"). When Δ_t is small, $\bar{A}_t \approx 1$ and the state is preserved ("ignore this input"). This selectivity makes Mamba effective for sequences with variable information density.

2.3 Our SSM Block

Each Mamba block in our implementation consists of:

1. **Layer normalization** on the input
2. **Input projection**: $[\mathbf{u}, \mathbf{g}] = \text{Linear}_{2H}(\mathbf{x}) \in \mathbb{R}^{2H}$
3. **Discretization**: $\Delta = \text{softplus}(\text{Linear}_H(\mathbf{u}))$
4. **Selective scan** (Eq. 7): $h_t = e^{A \cdot \Delta_t} \cdot h_{t-1} + (1 - e^{A \cdot \Delta_t}) \cdot u_t$
5. **Gated output**: $y_t = h_t \odot \sigma(g_t) + s \odot u_t$ (learnable skip s)
6. **Output projection**: $\text{Linear}_D(y_t)$ back to input dimension
7. **Residual connection**: output $= x_t +$ projected y_t

3 Connecting Mamba to GraphCast

3.1 Where to Insert Temporal Memory

The natural insertion point is between the encoder and processor, where the atmospheric state is represented as mesh-node features $\mathbf{m} \in \mathbb{R}^{N_{\text{mesh}} \times B \times D}$. At this point:

- The features are spatially structured (one vector per mesh node)
- The spatial extent is manageable (∼2,500 nodes for mesh_size=4)
- The downstream processor and decoder are unchanged

3.2 Failed Approach: Replace the Encoding Path

Our first design encoded each input timestep separately through the Grid-to-Mesh GNN, producing a 4D tensor $[\text{time}, \text{mesh}, B, D]$, then applied Mamba over the time axis.

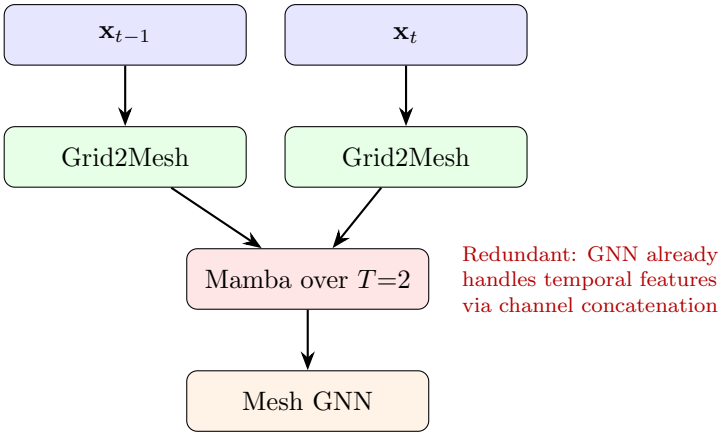

Figure 2: Failed approach: per-timestep encoding + Mamba. This changes the encoding path and makes Mamba redundant with the GNN's temporal handling.

Problems:

1. The encoding path differs from baseline (per-frame vs. concatenated), confounding comparisons.
2. Mamba processes only 2–4 frames—information the channel concatenation already captures.
3. Results: 15–20% worse than baseline.

3.3 Current Approach: Residual Memory

The corrected design preserves the baseline encoding path entirely and injects Mamba as a pure residual:

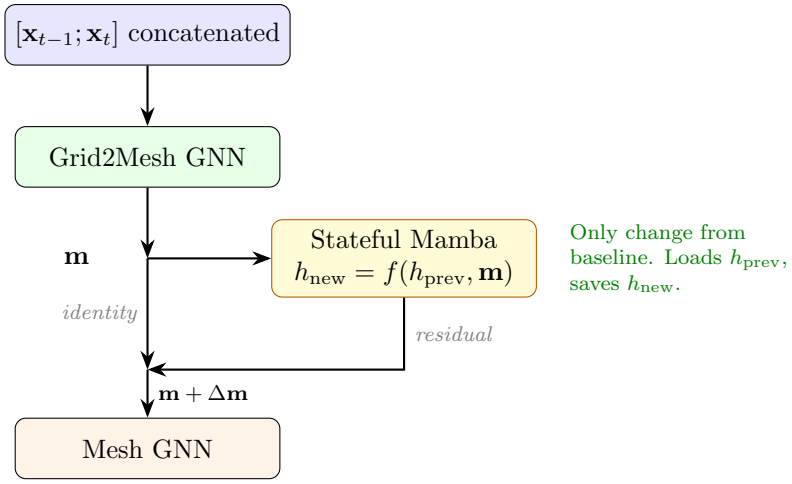

Figure 3: Residual memory: identical encoding to baseline, Mamba only injects cross-rollout state as a residual $\Delta\mathbf{m}$.

3.3.1 Cross-Rollout State Threading

During autoregressive rollout, the Mamba hidden state carries information across prediction steps:

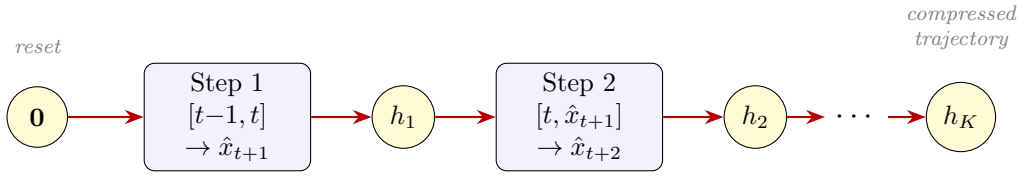

Figure 4: Hidden state h_k accumulates a compressed summary of the forecast trajectory. The baseline has no equivalent—each step is independent.

The three mechanisms enabling this:

1. `hk.scan` in GraphCast's autoregressive wrapper threads Haiku state between rollout steps (no code change needed).
2. `hk.get_state`/`hk.set_state` in the Mamba block loads the previous hidden state and saves the updated one.
3. `_reset_ssm_state()` in the training loop zeros all states before each new training sample, preventing cross-sample leakage.

3.4 Results So Far

Table 1: Residual memory vs. baseline (mesh_size=4, batch=1, input_steps=2, 6h eval on 2022)

Model	target_steps	RMSE @18k	Baseline @20k	Gap
Baseline	2	—	62.41	—
Residual Memory	2	64.77	62.41	+3.8%
Baseline	4	—	96.08	—
Residual Memory	4	105.21	96.08	+9.5%

The residual memory architecture does not yet outperform the baseline. The gap is much smaller than our earlier confounded approach (+3.8% vs. +16%), suggesting the encoding path was the primary culprit. However, the Mamba temporal memory itself is not providing enough value to offset its additional parameters and optimization cost.

4 Why Mamba Struggles in This Setting

1. **Information redundancy**: With T=2 input frames, the model already has access to the current state and its first-order tendency. The residual from Mamba's hidden state adds information from step $k-1$, but this is often similar to what the two input frames already encode.

2. **Capacity bottleneck**: The hidden state $h \in \mathbb{R}^{128}$ per mesh node must compress the entire atmospheric trajectory. The atmosphere has ∼1.4M scalar values per snapshot; $128 \times 2562 \approx 328$K hidden state parameters cannot store a meaningful fraction of the trajectory.

3. **Short training rollouts**: With target_steps=2 or 4, there are only 1–3 state-transfer opportunities. The gradient signal for learning what to store in h is weak.

4. **Optimization difficulty**: Gradients must flow through multiple autoregressive steps and the SSM recurrence, creating long credit-assignment chains.

5 Alternative Approaches for Temporal Memory

Given that Mamba has not yet succeeded, what other mechanisms could provide cross-rollout memory?

5.1 Approach 1: Transformer Cross-Attention over Rollout History

Idea: At each rollout step k, store the mesh latent $\mathbf{m}_k$ in a memory buffer. Use cross-attention to let the current step attend to all previous steps:

$$\mathbf{m}'_k = \mathbf{m}_k + \text{CrossAttn}(Q{=}\mathbf{m}_k,\ K{=}V{=}[\mathbf{m}_1, \ldots, \mathbf{m}_{k-1}])$$

Pros:
- Direct access to all previous states, no information bottleneck
- Attention can selectively focus on the most relevant past steps
- Well-understood optimization landscape

Cons:
- $O(K \cdot N_{\text{mesh}})$ memory to store history
- $O(K^2)$ attention cost per step (but K is small, e.g. 10)
- Requires positional encoding for the rollout step index

Implementation: Store mesh latents in a buffer that grows during rollout. Apply a small (1–2 layer) cross-attention module after the Grid-to-Mesh encoder.

5.2 Approach 2: Concatenate Previous Mesh Latent as Extra Input

Idea: Feed the mesh latent from the *previous rollout step* back as an additional input channel to the current step's mesh processor:

$$\mathbf{m}_k^{\text{input}} = [\mathbf{m}_k^{\text{encoded}}\ ;\ \mathbf{m}_{k-1}^{\text{processed}}]$$

This is analogous to how GraphCast already uses two atmospheric frames—we simply add a "previous mesh state" frame.

Pros:
- Extremely simple to implement (concatenation + linear projection)
- No new module types—uses existing GNN infrastructure
- Direct access to previous step's full representation (no bottleneck)
- Constant memory overhead (only store one previous step)

Cons:
- Only one step of explicit memory (but this propagates implicitly through the chain)
- Doubles the mesh latent input dimension to the processor

Assessment: This is arguably the simplest and most promising approach. It mirrors GraphCast's own temporal encoding strategy (channel concatenation) and avoids introducing any new module. The mesh latent is already a compressed, information-rich representation—feeding it back directly may be more effective than compressing it further into a 128-dimensional hidden state.

5.3 Approach 3: Auxiliary Loss on Temporal Consistency

Idea: Rather than adding an explicit memory module, add a loss term that encourages temporal consistency in the mesh latent space:

$$\mathcal{L}_{\text{temporal}} = \lambda \sum_{k=1}^{K-1} \left\| \frac{\mathbf{m}_{k+1} - \mathbf{m}_k}{\Delta t} - \frac{\mathbf{m}_k - \mathbf{m}_{k-1}}{\Delta t} \right\|^2$$

This penalizes "jerky" changes in the latent trajectory, encouraging smooth evolution and implicitly encoding temporal structure.

Pros:
- No additional parameters or modules
- Regularizes the latent space to be temporally coherent
- Easy to implement and tune via λ

Cons:
- Does not add new information, only constrains representations
- May conflict with the primary forecast loss
- Requires multi-step rollout during training (target_steps > 2)

5.4 Approach 4: Learned Error Correction Module

Idea: Train a separate lightweight network that takes the current prediction and an estimate of accumulated error, and outputs a correction:

$$\hat{\mathbf{x}}_{t+k\Delta t}^{\text{corrected}} = \hat{\mathbf{x}}_{t+k\Delta t} + \text{CorrectionNet}(\hat{\mathbf{x}}_{t+k\Delta t},\ k,\ \mathbf{e}_k)$$

where $\mathbf{e}_k$ is a running estimate of error characteristics (e.g., mean bias, spatial error pattern).

Pros:
- Directly addresses the error accumulation problem
- Can be trained on long rollouts even if the base model is frozen
- The correction can be conditioned on the rollout step index k

Cons:
- Requires ground truth at each rollout step for training
- Two-stage training (base model + correction) adds complexity
- May learn to patch over errors rather than prevent them

5.5 Approach 5: Larger Hidden State with Bottleneck Architecture

Idea: Keep the Mamba approach but address the capacity bottleneck. Use a much larger hidden state (H=1024 or 2048) with a bottleneck projection:

$$\mathbf{m}' = \mathbf{m} + \text{Linear}_{D \leftarrow H}(\text{Mamba}_H(\text{Linear}_{H \leftarrow D}(\mathbf{m}), h_{\text{prev}}))$$

Pros:
- Directly addresses the 128-dim capacity bottleneck
- Same architecture, just larger
- Bottleneck projection keeps the mesh latent dimension unchanged

Cons:
- Significantly more parameters and memory
- May still face the optimization difficulties of long-range credit assignment
- Does not address the fundamental question of whether temporal memory is useful

5.6 Comparison of Approaches

Table 2: Comparison of temporal memory approaches

Approach	Complexity	Memory	New Params	Key Advantage
Mamba residual (current)	Low	$O(N \cdot H)$	∼130K	Linear-time sequence processing
Cross-attention	Medium	$O(K \cdot N \cdot D)$	∼500K	Direct access to all past states
Concat prev. latent	Very low	$O(N \cdot D)$	∼33K	Simplest; mirrors existing design
Auxiliary loss	None	None	0	No new parameters
Error correction	Medium	Low	∼200K	Directly targets error drift
Larger Mamba	Low	$O(N \cdot H')$	∼2M	More expressive state

5.7 Recommended Next Step

We recommend **Approach 2: Concatenate Previous Mesh Latent** as the next experiment:

1. It is the simplest to implement (a few lines of code)
2. It provides full-resolution memory (no bottleneck)
3. It mirrors GraphCast's own temporal design principle
4. If it works, it validates the value of cross-rollout memory; if not, it strongly suggests the problem is elsewhere

The implementation requires only:

- Storing the mesh latent from the previous rollout step via `hk.get_state`/`hk.set_state`
- Concatenating it with the current mesh latent: $[\mathbf{m}_k; \mathbf{m}_{k-1}] \in \mathbb{R}^{N \times B \times 2D}$
- A single linear projection $\text{Linear}_{D \leftarrow 2D}$ to restore the dimension
- Zeroing the stored latent at the start of each training sample

6 Conclusion

GraphCast's memoryless autoregressive design is a clear architectural limitation for long-range forecasting. We attempted to address this with a Mamba SSM that carries hidden state across rollout steps. After correcting encoding path confounds, the residual memory architecture shows that Mamba does not significantly harm the model (gap reduced from −20% to −3.8%), but it does not help either.

The failure points to a specific bottleneck: compressing an atmospheric trajectory into 128 dimensions is too lossy to be useful. Alternative approaches that avoid this bottleneck—particularly direct latent concatenation (Approach 2) and cross-attention (Approach 1)—may be more effective. The simplest next experiment is to concatenate the previous rollout step's mesh latent as an additional input channel, which provides full-resolution memory with minimal code changes.

References

[1] R. Lam et al., "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023.

[2] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.